C64uRemote for M5Stack CoreS3

Technical Documentation

Version 1.0

Contents

Overview **2**

Hardware **2**
- Target platform . . . 2
- Peripherals . . . 3

Development environment **3**
- PlatformIO (recommended) . . . 3
- Arduino IDE 2.x (alternative) . . . 4
- Configuration (build_env.h) . . . 4

Wi-Fi subsystem **4**
- Runtime configuration instead of compile time . . . 4
- Screens and operation . . . 4
- NVS layout . . . 5
- Card format . . . 5
- /wifi.txt on the SD card . . . 5
- Setup portal . . . 5
- Direct mode . . . 6
  - Network number (since v1.5.0) . . . 7

Status lamp **7**
- Driving it . . . 7
- States . . . 7
- Behaviour during the upload . . . 7
- Settings . . . 8

Software architecture **8**
- State model . . . 8
- Rendering with an offscreen buffer . . . 8
- Battery indicator . . . 8

ReST connection to the C64 Ultimate **9**
- Endpoints used . . . 9
- Refused connections . . . 9
- Own HTTP path (since v1.4.0) . . . 9
  - NFC RF field . . . 10
- Reconnecting with several networks . . . 10
- CPU speed . . . 10
- Joystick ports . . . 10
- Disk images and autostart . . . 11
- Streaming upload . . . 11

NFC subsystem **11**
- Polling strategy . . . 11
  - Why a dedicated probe . . . 11
  - Returning to the main screen . . . 12
- Card types . . . 12

Command cards . . . 12
PowerOff with confirmation . . . 13
Writing cards . . . 13
Data format (TeensyROM/Zaparoo-compatible) . . . 13
NDEF parser: tolerance . . . 13
NFC-Info . . . 13
Copying (dump/restore) . . . 14

Control logic **14**
Touch handling . . . 14
Hit areas . . . 14
Scrollbar . . . 15
Start of the list window . . . 15
Button shortcuts that are gone . . . 15
PowerOff safeguard . . . 15

Persistence **15**

Project structure **16**

Troubleshooting **16**

Sources **16**

License **16**

How this was made **17**

Overview

C64uRemote is firmware for the M5Stack CoreS3 (ESP32-S3) that remotely controls the Commodore 64 Ultimate and Ultimate64 Elite-II via its ReST API. It also supports an RFID2 reader (NXP WS1850S) and a microSD card to launch programs from NFC cards and to manage cards.

Three things differ from the Core Basic edition: the device is **operated entirely through the touchscreen** (the CoreS3 no longer has buttons A/B/C), the image is built up in a **full-screen buffer in PSRAM**, and **Port A sits on different pins**. Everything else – ReST, NFC, SD browser, Wi-Fi setup – is functionally the same.

The firmware is implemented as a single translation unit (`src/main.cpp`, about 6000 lines) and uses the Arduino framework for ESP32.

Basis: Original project by Karl Prosser (github.com/ReadyOS-C64/C64uRemote), ports for M5StickC Plus2, M5Dial and M5Stack Core by Martin Oswald (1MHz.de); this CoreS3 version builds on those.

Hardware

Target platform

Property	Value
Board	M5Stack CoreS3 / CoreS3 SE / CoreS3 Lite
SoC	ESP32-S3 (dual-core Xtensa LX7, 240 MHz)
Flash / PSRAM	16 MB / 8 MB
Display	320 × 240, ILI9342C, via M5GFX
Controls	capacitive touchscreen (FT6336U), **no buttons A/B/C**
Hardware buttons	POWER and RESET on the side only

The CoreS3 has **no built-in NFC reader**. The card features need the Unit RFID2 on Port A – the same module as on the Core edition.

Peripherals

Device	Connection	Details
Unit RFID2	Port A (Grove, red)	I²C, SDA = GPIO 2, SCL = GPIO 1, address 0x28, 100 kHz
microSD	internal slot	SPI, SCK = GPIO 36, MISO = GPIO 35, MOSI = GPIO 37, CS = GPIO 4, FAT32
Status lamp SK6812	Port B (Grove, black)	Three LEDs chained (M5Stack Unit RGB), data line on the yellow wire = GPIO 9, 5 V from the port

Careful, a trap: on the CoreS3, Port A sits on G2/G1 – compared with the Core Basic (G21/G22) that is not only different pins, SDA and SCL are swapped as well. The internal I²C bus (G11/G12) with touch, PMU and RTC is unaffected.

Display and SD share the SPI bus. `SPI.begin()` is therefore called with the three pins explicitly, otherwise `SD.begin()` looks on the ESP32-S3 default assignment. SD initialization first tries 20 MHz, and 4 MHz on failure.

The status lamp hangs on a pure output line. **Whether an LED is connected cannot be detected** – an SK6812 has no return channel. If it is missing, the data goes nowhere; that has no side effect. There is therefore no detection for it, but a *Status-LED* switch in the setup. The ESP32-S3 puts out 3.3 V logic levels; the M5Stack units with SK6812 are designed for that.

Development environment

PlatformIO (recommended)

The configuration lives, version-controlled, in `platformio.ini`, which makes builds reproducible. Key settings:

Setting	Value	Reason
`board`	`m5stack-cores3`	CoreS3 / CoreS3 SE / CoreS3 Lite
`platform`	`espressif32`	Arduino framework
`board_build.partitions`	`huge_app.csv`	WiFi + SD + RFID exceed the standard app partition
`monitor_speed`	115200	serial output
`-DBOARD_HAS_PSRAM`		the offscreen buffer lives in PSRAM
`-DARDUINO_USB_CDC_ON_BOOT=1`		the CoreS3 enumerates as native USB CDC

Libraries (`lib_deps`):

- `m5stack/M5Unified` (≥ 0.2.8)
- `m5stack/M5GFX` (≥ 0.2.11)
- `bblanchon/ArduinoJson` (^6.21.5) — deliberately version 6, since version 7 no longer has `DynamicJsonDocument`
- `MFRC522_I2C` by kkloesener (Git) — I²C driver for the WS1850S
- `adafruit/Adafruit NeoPixel` (^1.12.0) — status lamp on Port B

If no port shows up while flashing: hold RESET for 3 s (green LED on, download mode) and start the upload again.

Commands:

```
pio run                 # compile
pio run -t upload       # flash
pio device monitor      # serial output
```

Arduino IDE 2.x (alternative)

Rename `main.cpp` to `C64uRemote.ino`, install the libraries manually and choose **Huge APP** under *Tools → Partition Scheme*.

Configuration (build_env.h)

`build_env.h` only supplies the **initial values** now. As soon as a network configuration is present in NVS, the file is ignored. Copy the template `build_env.h.example` to `build_env.h` and fill it in:

```
#define C64U_WIFI_SSID       "MyWiFi"
#define C64U_WIFI_PASSWORD   "MyPassword"
#define C64U_TARGET_HOST     "192.168.0.64"
#define C64U_TARGET_PASSWORD ""       // only if set on the C64
```

If the file is missing, the project compiles with empty defaults; the Core then shows *SETUP* > *WIFI* at startup and can be set up on the device.

Wi-Fi subsystem

Runtime configuration instead of compile time

SSID, password, c64u address and c64u password live in NVS at runtime and can be changed through *SETUP → WiFi*. Up to `kWifiProfileMax` (= 4) profiles are kept:

```
struct WifiProfile { String ssid; String pass; };
WifiProfile gWifiProfiles[kWifiProfileMax];
size_t      gWifiCount;
size_t      gWifiTry;      // profile for the next connection attempt
```

`beginWiFi()` takes `gWifiProfiles[gWifiTry]` and then advances the index by one. If a connection fails, `serviceWiFi()` tries the next profile after `kWiFiRetryMs` – across several rounds the attempt therefore walks through all known networks. At start-up `wifiPickBestProfile()` scans the area once and begins with the strongest known network.

`wifiAddProfile()` sorts a new network in at the front; a network that is already known merely gets a new password and moves to the front as well. The oldest profile drops off the back if need be.

Screens and operation

Five screens are added. They are operated like every other list page: **A** and **C** page through, **B** triggers, **B long** goes back.

Screen	Purpose
`WifiMenu`	Submenu with the eight entries
`WifiScan`	List of the networks found
`WifiCard`	Present a card, read the password
`WifiPortal`	Access point running; shows SSID, password, IP and time left
`WifiSaved`	Stored networks: connect, delete or write to a card

The settings list now has an `enum SettingsId` with `kSetWifi` as the last action. `kSetLastAction` separates actions from switches, and a `static_assert` keeps the names and the labels in step.

NVS layout

Namespace `c64unet`, separate from the interaction settings in `c64uremote`:

Key	Contents
`wn`	number of profiles (0…4)
`s0…s3`	SSID
`p0…p3`	password
`host`	address of the c64u
`hpass`	password of the c64u
`dmode`	direct mode on/off
`dssid`, `dpass`	SSID and password of the direct network
`dnet`	address range of the direct network, e.g. `192.168.4`

Factory Reset only touches `c64uremote`; the network configuration survives and is discarded exclusively through *WiFi → Delete all*.

Card format

Wi-Fi cards use the same scheme as Wi-Fi QR codes, stored as an ordinary NDEF text record:

`WIFI:S:<ssid>;T:WPA;P:<password>;;`

`parseWifiText()` evaluates the `S` and `P` fields, accepts backslash-escaped special characters and additionally understands the short form `WIFI:<ssid>;<password>`. `wifiCardText()` is the counterpart and is used by *WiFi → To NFC card*.

On the setup page (`ScreenMode::WifiCard`) a card text without the `WIFI:` prefix counts as a plain password for the network picked beforehand.

Outside the setup, `processCard()` recognises a complete `WIFI:` card, stores the network and connects right away. `gWifiTry` is pointed at that profile on purpose and the code waits up to `kWifiCardConnectMs` (8 s) for `WL_CONNECTED`; after that `serviceWiFi()` takes over again. If the network is already connected the function bails out with *ALREADY CONNECTED* – that prevents an endless loop when the card stays on the reader and the background poll spots it again.

/wifi.txt on the SD card

`loadWifiFromSd()` reads a simple key/value format. Every new `ssid` line begins an entry; `#` and `;` introduce comments. `host` and `hostpass` are recognised as well.

```
ssid = MyWiFi
pass = secret

host     = 192.168.0.64
hostpass =
```

The file is read at start-up (only while `gWifiCount == 0`) and on demand through *WiFi → Load from SD*.

`saveWifiToSd()` is the counterpart: it writes all profiles together with `host` and `hostpass` back to `/wifi.txt` with a comment header. An existing file is moved to `/wifi.bak` with `SD.rename()` beforehand; if that fails it is deleted. The function returns the number of networks written and puts a plain-text hint into `errorOut` on failure.

Setup portal

`startPortal()` switches to `WIFI_AP`, opens an access point on fixed channel 1 (`kPortalSsid` / `kPortalPass`), starts a `DNSServer` as a captive-portal redirect and a `WebServer` with two routes (`/` and `/save`). The station part is deliberately shut down: if it stayed active it would keep looking for the stored network in the background, change the radio channel while doing so and drop clients that had joined.

`servicePortal()` runs on every loop pass, holds the idle clock while a client is connected, and ends the portal after `kPortalIdleMs` (5 min) or `kPortalCloseMs` after a successful save. `stopPortal()` restores `WIFI_STA` and reconnects immediately.

Both classes come with the Arduino ESP32 core (`WebServer.h`, `DNSServer.h`); no extra libraries are needed.

Direct mode

For meetings without a router the CoreS3 opens a WiFi network of its own (`WIFI_AP`, without a station part – for the same reason as the setup portal: a searching STA changes channel and throws connected devices off).

SSID / password	`C64uRemote-Direct` / `c64ultimate` (defaults, changeable)
Channel, max. clients	6, 8
CoreS3 address	`<net>.1`
DHCP range	`<net>.64` … `<net>.74`
c64u target	`<net>.64`
`<net>`	`192.168.4` (default) or `192.168.2`, anything via `direct_net`

Start order: `WiFi.softAP()` first, then `WiFi.softAPConfig(ip, ip, 255.255.255.0, <net>.64)`. The fourth parameter sets the start of the DHCP range (Arduino core 2.0.17 hands out eleven addresses from there). The other way round, depending on the core version, the default `192.168.4.1` may stay – the setup portal never notices because it uses exactly that address.

Target address: in direct mode `targetHost()` returns `gDirectHost` instead of the stored home address, which stays untouched. If the device is connected as a normal client to a network whose SSID matches the direct network's, `<net>.64` applies as well – so a second remote can join without changing its settings.

Finding the c64u: the first DHCP lease is `.64`. If another device was quicker, `directNextCandidate()` keeps looking: candidates are `.64` and all addresses from `esp_wifi_ap_get_sta_list()` + `esp_netif_get_sta_list()`. After each failed status test the next one is tried. A device counts as the c64u if it answers `/v1/version` with JSON (or with 401/403 if a password is missing). Once an address has answered, it is only switched after the second failure in a row – the c64u occasionally refuses on its own (see *Refused connections*).

Status test: with nobody connected the test is skipped entirely. When a device joins (`WiFi.softAPgetStationNum()` rises) and the c64u is not confirmed yet, it is checked right away, starting again at `.64`. Since the c64u is often still busy with DHCP at that point, up to six follow-up checks follow at 3 s intervals (`directRecheckDue()`), on any page, until it answers. In the direct network the connection setup also has a 1 s deadline.

Resetting DHCP: the ESP32's DHCP server hands out addresses via a pointer that only moves forward. If the c64u releases its address when leaving or asks for the old one when coming back, the server drops the entry and hands out the next one (`.65`, `.66` …). So `directRestartDhcp()` restarts the server as soon as the last device has left – afterwards it starts at `.64` again. With other devices still connected this is skipped, because the server forgets all assignments and could otherwise hand out an address twice; the search then finds the c64u at its new address.

Switching: `setDirectMode()` stores the setting, discards the connection state and restarts AP or WiFi. An explicitly chosen network (list, scan, WiFi card) ends direct mode. The setup portal displaces the direct AP temporarily; after the portal, `beginWiFi()` starts it again.

Scanning in direct mode: only the station part can scan, so for the duration of the scan the mode switches briefly to `WIFI_AP_STA` and then back to `WIFI_AP`.

NVS (namespace `c64unet`): `dmode` (bool), `dssid`, `dpass`, `dnet`. Unusable values (SSID empty or > 32 characters, password < 8 or > 63, address range not three numbers 0…255) are replaced by the defaults on load.

Network number (since v1.5.0)

Several devices in direct mode at one table need different network names. The number lives solely in the network name `gDirectSsid` (NVS `dssid`): `directNumFromSsid()` returns 0 for `C64uRemote-Direct`, 1–9 for `C64uRemote-Direct-1` … `-9` and −1 for a freely set name. `directChannel()` picks the radio channel for `WiFi.softAP()` from it: without a number `kDirectChannel` (6) as before, otherwise 1, 6, 11 in turn. The menu entry *Direct no.* (`kWifiDirectNum`) steps 0 → 1 … 9 → 0 via `setDirectNum()`, saves and restarts a running access point.

Status lamp

Driving it

Three SK6812 chained on GPIO 9 (`kLedCount`), addressed through `Adafruit_NeoPixel` (on the ESP32-S3 the library uses the RMT peripheral; one `show()` for three LEDs takes under 100 µs). With fewer LEDs attached, those present simply take the first colours.

`serviceLed()` is called on every loop pass but only works every `kLedFrameMs` (20 ms), and only sends data when the colour has actually changed. On a static screen the pin therefore stays quiet.

States

Source	Colour
`!wifiConnected`	red
`!targetReachable`	blue
`!authOk`	yellow
everything ok	green
`portalActive`	magenta, running light at 400 ms per step
`screen == Busy`	cyan as a progress bar across the three LEDs
event flash	green or red on all three, fading out over `kLedFlashMs` (600 ms)

The progress bar works in integers: `done = 256 · kLedCount · busySent / busyTotal` gives the position in 256ths per LED. Sections below that shine fully, the one in progress is mixed from fill level and pulse, the rest stay dark. If `busyTotal == 0` (size unknown), a cyan running light at 160 ms per step takes over instead.

The precedence is in `serviceLed()`: flash before upload before portal before connection state. `ledFlash()` sets the colour and the expiry time and is called from `processCard()` as well as the write, dump and restore branches – green on success, red on every error exit.

`ledFlashMs == 0` explicitly means “no flash”. Without that marker the old timestamp would stay put, and the rollover comparison `(int32_t)(now - ledFlashMs) < 0` would become true again after roughly 25 days of uptime - the lamp would then show the old flash colour for weeks.

`ledPulse()` produces a triangle curve 0…255…0 without floating point and without a table; `ledScale()` dims a colour proportionally.

Behaviour during the upload

The streaming upload blocks the main loop. So that the lamp still pulses, `publishProgress()` – which is called per block anyway – additionally calls `serviceLed()`. The same goes for `waitCursorBlinking()`, which waits up to 15 s after mounting a disk image.

Settings

`ledEnabled` and `ledBrightness` live in NVS (`led_on`, `led_br`). The brightness steps (`nextLedBrightness`) are finer at the bottom end than for the display, because an SK6812 is already dazzling at low values. When the lamp is switched off, `serviceLed()` sends black once and nothing after that.

Software architecture

State model

The entire runtime state lives in the global `struct AppState app`. The display follows a screen enum:

```
enum class ScreenMode {
  Home, CpuMenu, Status, Settings, SdBrowser,
  RfidRun, RfidWrite, RfidInfo, RfidDump, RfidRestore, Busy
};
```

The main loop `loop()` runs at ~30 fps (`kFrameMs = 33`):

1. `M5.update()` — update touch and timers
2. `servicePortal()` / `serviceWiFi()` / `refreshConnectionStatus()` — maintain the network
3. `handleTouch()` — evaluate touch input
4. `serviceRfid()` — RFID polling (every 250 ms, only on RFID screens)
5. `updateHomeDemo()` — animation sequencing
6. `render()` — update the display

Rendering with an offscreen buffer

Unlike the Core Basic, the CoreS3 has 8 MB of PSRAM. `setup()` therefore creates a full-screen sprite:

```
canvas.setPsram(true);
canvas.setColorDepth(16);
gUseCanvas = canvas.createSprite(kScrW, kScrH);   // 320 × 240 × 2 = 150 kB
gDraw      = gUseCanvas ? &canvas : &M5.Display;
```

All drawing goes through `gDraw`. If the allocation fails, `gDraw` points at the display and `pushFrame()` becomes a no-op – the code then behaves like the Core edition, just with a visible build-up.

The buffer keeps its contents between two frames, so the partial updates (status bar only, tiles only, effect area only) keep working unchanged. `render()` records in `drawn` whether anything was drawn at all and only then calls `pushFrame()`: one `pushSprite()` costs 150 kB across the same SPI bus the SD card hangs on.

The logo lives in flash (`1MHz_logo_rgb565.h`, 240 × 135, RGB565) and is read via `pgm_read_word`. Additional RAM required: one line buffer (`rowBuf`, 640 bytes) and two scaling tables (`logoXMap`, `logoYMap`, together ~1 kB).

The effect computation (`drawDistortedRows`, `drawRotoZoom`, `drawRipple`, `drawRasterBars`) scales via `FX Detail`: with “Half” only every second pixel and every second line is computed and doubled on output (four times less compute). Menus are redrawn only on change (`screenDirty`, `barDirty`) so nothing flickers.

Battery indicator

The level comes from M5Unified: `M5.Power.getBatteryLevel()` (0-100, negative = no battery), `M5.Power.isCharging()` and `M5.Power.getVBUSVoltage()`. It is read at most every `kBattPollMs` (5 s); blinking and swapping run on the remembered value and cost no further I2C traffic.

The CoreS3 has an **AXP2101**. It provides a real percentage and measures the VBUS voltage, so the firmware also detects the cable being plugged in.

Drawing happens in three places:

- `drawBatteryLine()` replaces the line below the bar. The base line stays `kColLine`, the filled part is two pixels tall. At 0 % it would be too narrow to see, hence `if (width < 4) width = 4;`.
- `drawBatterySymbol()` draws the frame (32 x 12 px) and the terminal and puts the percentage in the middle. The frame is deliberately not filled - the line does that, and the number stays readable.
- `drawChargeBolt()` puts a 5 x 7 pixel bolt to the left of it, drawn row by row from a small table instead of from triangles; at this size the shape would not hold otherwise.

The colour for all three comes from `batteryColor()`, the thresholds are `kBattGreen`, `kBattYellow` and `kBattBlinkAt` at the top of the file.

`drawStatusBar()` used to run at most once per second - too rarely for visible blinking. On the lowest step `render()` therefore shortens the interval to `kBattBlinkMs` (500 ms).

ReST connection to the C64 Ultimate

All commands go through the HTTP ReST API of the Ultimate firmware (from 3.11). Base URL: `http://<host>/v1/…`. If a network password is set, it is sent in the `X-Password` header.

Endpoints used

- **Version / reachability** — `GET /v1/version`
- **Reset** — `PUT /v1/machine:reset`
- **Reboot** — `PUT /v1/machine:reboot`
- **Power off** — `PUT /v1/machine:poweroff`
- **Ultimate menu** — `PUT /v1/machine:menu_button`
- **Write memory** — `PUT/POST /v1/machine:writemem?address=…`
- **Read memory** — `GET /v1/machine:readmem?address=…&length=…`
- **Config categories** — `GET /v1/configs`
- **Read/set config** — `GET/PUT /v1/configs/<category>/<item>`
- **Start PRG** — `POST /v1/runners:run_prg`
- **Start CRT** — `POST /v1/runners:run_crt`
- **Play SID/MOD** — `POST /v1/runners:sidplay` / `:modplay`
- **Mount disk** — `POST /v1/drives/<a|b>:mount?type=…&mode=…`
- **Drive on** — `PUT /v1/drives/<a|b>:on`

`sendApiRequest()` wraps GET/PUT over an own HTTP path (see below) and parses the JSON response with ArduinoJson (`errors` array).

Refused connections

The HTTP server of the Ultimate firmware accepts only one connection at a time and refuses further ones with a TCP RST; `HTTPClient` reports this as *connection refused*. It was observed with only a single device on the network as well - so it happens sporadically and is neither a radio nor an address problem.

The actual call lives in `sendApiRequestOnce()`. `sendApiRequest()` is a wrapper around it: if no connection comes about (`HTTPC_ERROR_CONNECTION_REFUSED`), up to two more attempts follow after `kApiRetryDelayMs` (`kApiConnectAttempts` = 3). Retrying happens **only** in this case - nothing reached the c64u then, so a command cannot be doubled. Read errors and HTTP error statuses (4xx, 5xx) are passed through unchanged.

On top of that `refreshConnectionStatus()` saves a request: without a stored password the second query would be byte-identical to the first, because the `X-Password` header is only set when there is one. That halves the base load on the c64u.

Own HTTP path (since v1.4.0)

All requests to the c64u - `sendApiRequestOnce()`, `readC64Byte()` and the uploads in `uploadFile()` - no longer go through `HTTPClient`/`WiFiClient` but through small functions on lwIP sockets:

- `rawOpen()` connects without blocking and checks via `select()` without waiting every 2 ms; limit `kHttpConnectTimeoutMs` (1.5 s), in direct mode `kHttpConnectDirectMs` (1 s).
- `rawSendAll()` sends the request or file in pieces with `MSG_DONTWAIT`.
- `rawReadResponse()` reads the reply until the c64u closes the connection (or shortly after the last byte announced by `Content-Length`) and evaluates the status line, `Content-Length` and `chunked`. If the c64u closes without a reply the error is *connection lost*, if the reply does not come, *read Timeout*.

`rawHttpRequest()` combines this for GET/PUT; every request carries `Connection: close`. Background: with `HTTPClient` most requests got lost on the home network at times, while a connection checked without waiting got through reliably.

`staSsid()` and `staRssi()` return network name and signal strength from a cache that is refreshed at most once per second via `esp_wifi_sta_get_ap_info()`. `WiFi.SSID()` asks the driver on every call, and `targetHost()` needs the network name all the time (detecting the direct network as a client). `directStationCount()` asks for the number of stations in direct mode at most every 500 ms.

NFC RF field

The MFRC522 driver leaves the 13.56 MHz field on permanently after `PCD_Init()`. `rfidFieldOn()` and `rfidFieldOff()` switch it on only for the card probe (`cardPresent()`, `cardPresentQuick()`, then 5 ms for the card to power up) and while a detected card is being processed; on an empty probe it goes off again. It is also switched off before every network request (`rawOpen()`) - according to M5Stack the M5Dial's RFID and WiFi share the antenna, and WiFi is blocked while the field is on. So that a card left on the reader is not executed or written again every time the field comes on, `rfidHoldCard()` remembers its UID after `processCard()`. `rfidHeldCardGone()` switches the field on briefly on every check, reads the UID and switches off again right away; the same card is ignored, only two misses in a row or a different card clear the way. On the M5Dial, whose NFC antenna sits right next to the WiFi antenna, the permanently switched-on field disturbed WiFi reception; here switching it off mainly saves power.

Reconnecting with several networks

`beginWiFi()` moves on to the next stored profile after every attempt. Without a countermeasure that means: with two networks stored of which only one is reachable, every other reconnect after a dropout hits the dead one and costs a full retry cycle (`kWiFiRetryMs`, 10 s).

`serviceWiFi()` therefore remembers the profile the connection came up on, via `wifiProfileIndex(staSsid())`, and makes it the next attempt; `gWifiNoted` keeps this to once per connection. After a dropout the first attempt goes back to the working network, and the dead one is only tried if the good one is really gone.

CPU speed

The path to the CPU-speed item is not fixed but discovered: first in "U64 Specific Settings", otherwise across all categories (`resolveCpuPath`). The available values come from the item's `values` array; a fixed list serves as a fallback (`setFallbackCpuChoices`).

Joystick ports

There is no `machine:` command for swapping the joystick ports. The mapping is a configuration item - in testing *Joystick Swapper* in the category *U64 Specific Settings*, with the values `Normal`, `Swapped`, `WASD Port 2` and `WASD Port 1`. It is therefore set through `/v1/configs/<category>/<item>?value=…`, exactly like the clock speed.

As with the clock, `resolveJoyPath()` looks in *U64 Specific Settings* first and otherwise walks all categories until an item name contains "Joystick", so a later firmware renaming it goes unnoticed. `refreshJoyChoices()` reads `values` and `current`.

`joyTokenFromValue()` and `joyValueFromToken()` convert between the device value and the card token (`WASD Port 1` <-> `WASD1`), so a card needs neither a blank nor a firmware-specific spelling.

`toggleJoystickSwap()` toggles between `Normal` and `Swapped` and returns to `Normal` from a WASD mode; `cycleJoystickValue()` walks through every reported value in the setup list.

Disk images and autostart

When mounting, the target drive is determined (`resolveTargetDrive`): with "Auto" the Core searches via `GET /v1/drives` for the drive with `bus_id: 8`. After mounting comes `:on`, then depending on *Disk Action* a reset and an autostart.

The autostart types via DMA into the C64 keyboard buffer (`writemem` at `$0277` ff., count register `$C6`). It uses the abbreviated BASIC commands `lO"*",8,1` and `rU`, so the line fits into the ten bytes of the buffer.

The wait times are **not fixed** but governed by `$CC` (BLNSW, cursor blink): `waitCursorBlinking()` polls `readmem` and detects from the blinking when BASIC is ready and when loading has finished. Load timeout: 3 minutes.

Streaming upload

Since large `.d64` files (up to ~800 kB for `.d81`) do not fit in RAM, `uploadFile()` sends the file in blocks (1 kB) directly from the SD stream into a `WiFiClient` socket. Multipart boundaries and headers are built manually; the `type/mode` arguments go in the URL, not as a form field (otherwise the firmware interprets every multipart part as a file).

NFC subsystem

Polling strategy

Which screens the background poll runs on is decided by `autoNfcScreen()`. This used to be the home screen only - in the SD browser a card on the reader went unnoticed. Allowed now are `Home`, `Status`, `CpuMenu` and `SdBrowser`, the last one only with `sdPickMode == 0`: if the user is picking a file for *NFC-Write* or a dump to restore, a card must not start anything.

`app.autoRfidFrom` remembers the screen it came from; after `kAutoRfidHoldMs` the read page returns exactly there instead of always to the home screen. The back field does the same.

`restoreAutoRfidScreen()` restores two things by hand that would otherwise be lost:

- `setScreen()` resets `gListStart` - without a backup the browser would sit at the very top again after every card.
- A directory or random card makes `resolveRandomFile()` read a different directory, rebuilding `app.sdPath` and the file list in the process. If the path differs, the original directory is read back in.

If the read page is opened by hand through the `RFID` tile, `activateTile()` explicitly sets `autoRfidFrom` to `Home` - otherwise the return path from the last automatic call would linger.

`serviceRfid()` runs in two modes:

1. **On the RFID screens** (`RfidRun`, `RfidWrite`, `RfidInfo`, `RfidDump`, `RfidRestore`) a full `cardPresent()` poll happens every 250 ms.
2. **On the main screen** a quick probe with `cardPresentQuick()` runs at the interval configured under *Auto-NFC*. When it finds a card, the code sets `autoRfidActive`, switches to `RfidRun` through `setScreen()`, draws one `render()` and then calls the same handling the reading screen uses.

The actual card handling lives in `processCard()`. Both paths call it, so there is only one implementation.

Why a dedicated probe

With no card present, the MFRC522 waits after the REQA command until its internal timer expires. `PCD_Init()` sets that to 0x03E8 = 1000 steps of 25 µs, i.e. 25 ms. The main loop stalls for exactly that long, which would show up as a stutter while an animation is running.

A card answers far more quickly: the frame delay time at 106 kbit/s is roughly 86 µs. `cardPresentQuick()` therefore shortens the window to about 2 ms for the probe only and restores it immediately afterwards — before `PICC_ReadCardSerial()`. Selection, authentication and every write operation still run with the full window.

```
void setRfidTimerReload(uint16_t ticks) {          // 1 tick = 25 µs
  rfid.PCD_WriteRegister(MFRC522_I2C::TReloadRegH, ticks >> 8);
  rfid.PCD_WriteRegister(MFRC522_I2C::TReloadRegL, ticks & 0xFF);
}
```

The original value is not hard-coded but read from `TReloadRegH/L` in `initRfid()` and kept in `gRfidTimerReload`. Should a future library version change the default, the behaviour stays correct.

Cost. A probe that finds nothing consists of a handful of I²C register accesses plus the roughly 2 ms wait, some 5 ms in total. At the default interval of 700 ms that is a background load below one per cent, not enough to miss a 33 ms frame.

Auto-NFC	Interval	approximate load
Off	–	0 %
1.5s	1500 ms	~0.3 %
0.7s	700 ms	~0.7 %
0.3s	300 ms	~1.7 %

Returning to the main screen

A reading screen opened automatically falls back after `kAutoRfidHoldMs` (20 s), leaving time to present further cards. The flag `app.autoRfidActive` distinguishes it from a screen the user selected deliberately, which stays put. Any button press and any manual navigation clear the flag.

Card types

Family	Detection	Access
MIFARE Classic 1K/4K/Mini	SAK	16-byte blocks, authentication required
NTAG213/215/216, Ultralight	SAK 0x00	4-byte pages, no key

`cardKind()` distinguishes by the SAK byte. The exact NTAG type comes from the `GET_VERSION` command (0x60), or as a fallback from the Capability Container (page 3).

Command cards

If a card carries the prefix `CMD:` instead of a file path, its content is executed as a command for the c64u. Neither an SD card nor a file is needed.

```
CMD:RESET
CMD:REBOOT
CMD:MENU
CMD:POWEROFF=0      power off immediately
CMD:POWEROFF=8      ask first, 8 s confirmation window
CMD:POWEROFF        ask first, using the device setting "NFC-Cmd PowOff"
CMD:M5OFF           switch the CoreS3 itself off (also CMD:DIALOFF)
CMD:CPU=10          set the CPU to 10 MHz
CMD:JOY             toggle the joystick ports (Normal <-> Swapped)
CMD:JOY=SWAPPED     set the ports fixed; also NORMAL, WASD1, WASD2
```

`parseCardCommand()` takes the text apart: check the prefix, split off an optional argument after `=`, compare the keyword in upper case. Whitespace and letter case do not matter. The content stays a plain NDEF text record, so any NFC app can read and write such a card.

In its reading branch `processCard()` checks for a command first and calls `runCardCommand()`. If a card carries the prefix but no known keyword, the device reports *BEFEHL UNBEKANNT* instead of turning it into a file path.

PowerOff with confirmation

The waiting time is an argument **on the card**, not in the device; `cardPowerOffSeconds()` returns it. Without an argument the setting *NFC-Cmd PowOff* applies (3/5/8/15 s), `0` means "no prompt".

The sequence uses three fields in `AppState`:

```
bool     cardPowerOffPending;
String   cardPowerOffUid;      // only the same card confirms
uint32_t cardPowerOffUntilMs;
```

Confirmation happens by presenting the same card again or by pressing the button. A different card cancels, and so does letting the window expire - both clear the flag without triggering anything. Binding to the UID prevents an unrelated card placed nearby from powering the machine down.

Writing cards

`cmdListAt()` builds the selection list: five fixed commands, then every CPU step currently held in `app.cpuDisplayOptions` (loaded from the c64u, otherwise the built-in fallback list). `cardCommandText()` turns the choice into the card text. The screen `ScreenMode::CmdPick` shows the list; the selection ends up in `app.pendingCardText`, which the writing screen prefers over `pathToCardText(app.pendingPath)`.

Data format (TeensyROM/Zaparoo-compatible)

The card holds a single **NDEF record of type Text** (Well Known, UTF-8). The content is the path to the program file:

```
SD:OneLoad v5/Bubble Bobble.crt
```

Prefixes `SD:`, `USB:`, `TR:` are accepted (the latter two are searched on the SD). A `?` as the filename, or a directory path, launches a random file from the folder. Maximum text length: 246 characters.

Storage:

- NTAG/Ultralight: NDEF TLV from page 4, pages 0–3 remain untouched.
- MIFARE Classic: NDEF TLV in the data blocks from block 4, trailers are skipped. Authentication first with the NDEF key `D3F7D3F7D3F7`, otherwise the factory key `FFFFFFFFFFFF`.

NDEF parser: tolerance

`parseNdefText()` reads the text robustly. Important edge cases:

- **Wrong payload length:** the TeensyROM writes a constant `0x10` into the length byte, even though the TLV length is correct. For the last record (ME flag) the TLV length therefore takes precedence, otherwise the path would be truncated.
- Filler bytes (`0x00`) and the TLV terminator (`0xFE`) end the text.
- Multiple slashes (`SD://folder`) are collapsed.
- Leading Lock-Control TLVs and 3-byte lengths are skipped.

For writing, `buildNdefText()` produces a clean short record. The former raw format `C64UPATH` is still recognized when reading.

NFC-Info

`collectCardInfo()` fills two pages (switchable on the device with A/C):

- **Page 1:** UID, SAK, type, memory size, version/manufacturer, content, path, filename and a check against the SD card.

- **Page 2:** hex dump of pages 0–15, lock bytes, decoded Capability Container, password protection from the config page and the NFC read counter.

Commands that not every card supports (`GET_VERSION`, config page, `READ_CNT`) deliberately run last, since an unsupported command deselects the card. After a failed `GET_VERSION` the card is made responsive again via `reselectCard()` (WUPA + Select). A read card is read only once per UID and then buffered, so the display is not rebuilt on every poll.

Copying (dump/restore)

`dumpCardToSd()` writes the card content as a text file to `/NFC-DUMPS/<uid>.nfc`. For MIFARE Classic a **key dictionary** (13 common keys) is tried per sector; the key found is noted as a comment and inserted into the trailer line (since key A always reads back as `00…`).

`restoreDumpToCard()` writes back the data blocks and – with valid access bits – the sector trailers as well. Not written are:

- **Block 0** (UID) — write-protected on normal cards.
- **Key A** (trailer bytes 0–5) — fundamentally not readable.
- Trailers with **invalid access bits** — `validAccessBits()` checks the complement encoding to prevent permanently locking the sector.

Data blocks are always written before the corresponding trailer, so access within the sector is not lost.

Control logic

Touch handling

`handleTouch()` is a small state machine on top of `M5.Touch.getDetail()`:

1. **Touch down** records position, timestamp and the current start of the window. If the finger lands on the scrollbar, the view jumps to that spot immediately and the thumb sticks to the finger.
2. **Staying down** pushes the list along with the finger on list pages: the start of the window follows from `startList - rows`, where `rows` is derived from the distance with **rounding**. The movement counts as a push exactly when `rows != 0` – that is, from half a row height (13 px) on. Once pushed it stays pushed, even if the finger travels back again.
3. **Lift off** only produces a tap if neither a push nor a scrollbar drag happened, the screen has not changed, and the finger has not travelled further than `kTapSlopPx` (10 px). From `kLongTapMs` (600 ms) it counts as a long tap.

The second point is the heart of the matter: previously nothing happened until a fixed swipe distance of 34 px had been covered, and then the list jumped three rows at a time. A slow, short swipe stayed below that and was counted as a selection on lift-off.

What matters here is that there is **no separate pixel threshold** for the start of the push. If it sat below one row height, a dead zone would open up in between: the list does not move yet, but the tap is already suppressed. The condition is therefore the same as the visible effect.

If a list fits on the screen entirely, no pushing happens at all – otherwise a wobble would swallow the selection on short lists (four CPU steps, say).

The screen check in point 3 catches an unpleasant case: if the background poll opens the read page while the finger is still down, lifting off would otherwise arrive as a tap on the new page – and in the worst case would have confirmed a PowerOff prompt there.

`handleTap()` distributes the tap: first to the bottom bar (back field on the left, two paging fields on the right), then depending on the screen to tiles (`tileFromTouch`) or list rows (`listRowFromTouch`).

Hit areas

Area	Range
Status bar	y 0 … 19
Logo / effects	y 20 … 127
Tiles, row 1 (5 of them)	y 132 … 164
Tiles, row 2 (4 of them)	y 168 … 200
List rows (6 × 26 px)	y 46 … 201, x 8 … 302
Scrollbar	y 46 … 201, x ≥ 298 (drawn from 304)
Back	y ≥ 204, x < 78
Paging ▲ / ▼	y ≥ 204, x 228 … 273 and 274 … 319

At 36 px the bottom bar is twice as tall as before, and the row height has grown from 24 to 26 px; in exchange only six rows are visible instead of seven. `tileFromTouch()` uses the same `tileRect()` as `drawTiles()`, so the display and the hit area cannot drift apart.

Scrollbar

As soon as there are more entries than rows fit, `drawScrollbar()` draws a bar to the right of the list. The height of the thumb is proportional to the visible share but never smaller than `kThumbMinH` (28 px) – otherwise long lists would make it impossible to hit. The hit area `hitScrollbar()` deliberately reaches 6 px further left than the bar that is drawn.

`scrollbarSeek()` converts a touch into a start of the window and centres the thumb under the finger. Touching down and dragging use the same function, so jumping to a spot flows seamlessly into dragging.

Start of the list window

`gListStart` is the first visible row and the single source of truth about the section on screen. In the button edition the window was derived from the selection and always centred on it – on a touchscreen that does not work, because the user pushes the list without selecting anything; the window would jump straight back to the selection.

`listWindowStart()` therefore only clamps to the valid range. The view is pulled along explicitly through `listEnsureVisible()` when the selection itself moves – that is, through the paging fields of the bottom bar. `setScreen()` and every directory change reset `gListStart` to 0.

Button shortcuts that are gone

The four freely assignable actions of the Core edition (`ShortcutAction`, B long, C long, B long + C, B long + A) are gone for good, as are `Kombi Zeit` and `PowerOff Kombi`. Without buttons there would be no gesture for them, and every command is a tile on the home screen anyway. `SettingsId` and `kSettingsItems` have shrunk accordingly from 26 to 20 entries.

PowerOff safeguard

- **POWER tile:** a second tap on the same tile within `PowerOff Zeit`. A tap on a different tile or next to the tiles cancels and redraws them so the warning colour disappears.
- **PowerOff card:** `CMD:POWEROFF=<seconds>` opens a time window of its own (`NFC-Cmd PowOff`) in which either the same card is presented again or the screen is tapped.

Persistence

Settings live in NVS (`Preferences`, namespace `c64uremote`). Time values are stored in tenths of a second and clamped to the valid range on load. `loadDefaultSettings()` restores the factory defaults.

The Wi-Fi credentials live in a namespace of **their own**, `c64unet` (see the *Wi-Fi subsystem* chapter), so they survive a *Factory Reset*. `build_env.h` only supplies the initial values as long as nothing has been stored there.

Project structure

```
M5CoreS3_C64uRemote/
├── platformio.ini            board, libraries, partition, upload
├── README.md
├── LICENSE                   MIT - Karl Prosser, Martin Oswald
├── wifi.txt.example          template for /wifi.txt on the SD card
├── .vscode/                  recommended extensions, editor settings
├── docs-src/                 markdown sources of the manuals
├── doc/                      finished manuals (PDF)
├── src/                      German edition
│   ├── main.cpp              entire firmware
│   ├── build_env.h           credentials (do not version)
│   ├── build_env.h.example   template
│   └── 1MHz_logo_rgb565.h    logo as an RGB565 array in flash
└── src-en/
    └── main.cpp              English edition, identical code
```

Troubleshooting

The serial monitor (115200 baud) prints heap, RFID and SD status on startup. When reading a card it logs the text read, for rejected files the path and extension, and for a disk mount the full URL. For on-device NFC analysis use NFC-Info page 2.

Typical messages:

Message	Cause
`SET build_env.h`	credentials missing
`NO WIFI`	no Wi-Fi connection
`AUTH?` (status bar)	network password set on the C64, not stored here
`TYP UNBEKANNT: …`	file extension not supported by the Ultimate
`KARTE OHNE C64U-PFAD`	card contains no recognizable path
`LADEN DAUERT ZU LANGE`	disk autostart after 3 min without READY

Sources

- ReST API of the Ultimate firmware: 1541u-documentation.readthedocs.io/en/latest/api/api_calls.html
- TeensyROM NFC Loader (card format): github.com/SensoriumEmbedded/TeensyROM/blob/main/docs/
- Reference implementation ultimate64 (Rust): github.com/mlund/ultimate64
- MFRC522_I2C library: github.com/kkloesener/MFRC522_I2C
- Original project C64uRemote: github.com/ReadyOS-C64/C64uRemote

License

C64uRemote is released under the **MIT License**. The full text is in the file `LICENSE` in the project root.

It originates from **C64uRemote by Karl Prosser (@klumsy)**, https://github.com/ReadyOS-C64/C64uRemote, which he published under the MIT License. This version is a derivative work and is released under the same terms:

- Copyright (c) 2026 Karl Prosser – original project
- Copyright (c) 2026 Martin Oswald (@mad, https://1MHz.de) – port and extensions

The MIT License allows you to use, modify and redistribute the software, including commercially. The only condition: **the copyright notice and the license text must be kept** and included with every copy. There is no warranty and no liability.

The libraries used carry their own licenses: M5Unified and M5GFX (MIT, © M5Stack), ArduinoJson (MIT, © Benoit Blanchon) and MFRC522_I2C (https://github.com/kkloesener/MFRC522_I2C). PlatformIO fetches them at build time.

How this was made

The port, the extensions and these manuals were written with the help of Claude (Anthropic). Concept, idea, hardware decisions and every test on real devices: Martin Oswald (@mad).